

CADERNO DE PROBLEMAS

30/10/2024 – 14h às 17h30

Leia atentamente estas instruções antes de iniciar a resolução dos problemas. Este caderno é composto de 6 problemas, sendo que 2 deles estão descritos em inglês.

1. Não é permitido o uso de material impresso, livros, apostilas e dicionários. Apenas é permitido o uso de lápis, caneta, lapiseira, borracha, régua e papel para rascunho (o papel é fornecido pela comissão organizadora). O acesso à internet é bloqueado durante a realização da prova. A ajuda do ambiente de desenvolvimento como C, C++, Java ou Python pode ser utilizada.

2. A correção das resoluções dos problemas será automatizada, por meio do sistema de submissão eletrônica BOCA, tendo como base os resultados obtidos a partir de uma série de execuções dos algoritmos submetidos pelos times.

3. Siga atentamente as exigências da tarefa quanto ao formato da entrada e da saída do seu algoritmo. Não implemente nenhum recurso gráfico nas suas soluções (janelas, menus, etc.), nem utilize qualquer rotina para limpar a tela ou posicionar o cursor.

4. Os problemas não estão ordenados por ordem de dificuldade neste caderno. A sugestão é de que procure resolver primeiro os problemas mais fáceis.

5. Utilize os nomes indicados nos problemas para nomear os seus arquivos-fonte, de acordo com a linguagem de programação utilizada.

6. Os problemas devem ser resolvidos utilizando o raciocínio entrada-processamento-saída, ou seja, não é necessário armazenar toda a saída para depois exibi-la.

7. Dicas de leitura (entrada de dados) e exibições (saída de dados) encontram-se no verso desta folha.

Faculdade de Informática de Presidente Prudente

http://www.unoeste.br/fipp/maratona

Observações:

Não utilize arquivos para entrada ou saída. Todos os dados devem ser lidos da entrada padrão (teclado) e escritos na saída padrão (tela). Utilize as funções padrão para entrada e saída de dados, como os exemplos a seguir:

- em C: `scanf`, `getchar`, `printf`, `putchar`;
  Para ler um **char** use: `scanf(" %c",&c);` não use: `fflush(stdin); scanf("%c",&c);`

- em C++: as mesmas de C ou os objetos `cin` e `cout`;
  Em C ou C++: use `sprintf(str,"%d",num);` em vez de `itoa(num,str,10);`

  Em C ou C++: não use `include<conio.h>`

- em Java, pode-se usar a classe `Scanner`:

```
Scanner sc = new Scanner(System.in);
int i = sc.nextInt();
String s = sc.next();
float f = sc.nextFloat();
```

  Ou ainda as classes `InputStreamReader` e `BufferedReader`:

```
InputStreamReader isr = new InputStreamReader(System.in);
BufferedReader br = new BufferedReader(isr);
try {
    v = br.readLine();
} catch(Exception e){  }
```

  Para exibição:

```
System.out.println();
System.out.printf("%d", i); //também %f, %c, %s, ...
```

  Em Java: não utilize `package`, o arquivo fonte deve ter a classe com os métodos e o `main`.

- em Python:
  Para leitura de um **int** use: `i = int(input())`
  Para leitura de um **float** use: `f = float(input())`
  Para leitura de um **char** use: `c = input()[0]`
  Para leitura de valores sequencias (quantidade fixa): `10 20 30`

```
v1, v2, v3 = list(map(int, input().split()))
```

  Para leitura de valores sequencias (quantidade não fixa): `10 20 30 ....`

```
valores = list(map(int, input().split()))
for v in valores:
    print(v) # faz alguma coisa com os itens da lista
```

  Para leitura de várias strings (quantidade não fixa): `aaa bbbb ccc`

```
strings = list(map(str, input().split()))
for s in strings:
    print(s) # faz alguma coisa com as strings da lista
```

Contando o Caos **(amarela)**

Problema
A

Arquivo fonte: *A.c*, *A.cpp*, *A.java* ou *A.py*

O rival de Wolfgang Puck, o Emeril Lagasse ("BAM!"), recentemente estabeleceu o recorde mundial na categoria de menor suflê, medindo apenas 2 cm! Wolfgang, para não ficar para trás, decidiu que ele estabeleceria um recorde culinário próprio: o bolo mármore mais simétrico do mundo. Isso claramente não é uma tarefa fácil!

Como todos sabemos pela biografia de Wolfgang, ele é um chef muito supersticioso. Em suas tentativas de criar o bolo simétrico, ele jurou remover o bolo do forno somente em um horário palíndromo, ou seja, um horário que lido da esquerda para a direita é o mesmo que lido da direita para a esquerda.

Não incluindo o horário atual, quando será a próxima oportunidade para Wolfgang remover seu bolo?

Entrada

Na primeira linha da entrada você recebe *n*, o número de tentativas que Wolfgang faz para fazer seu bolo simétrico. As *n* linhas seguintes contêm uma string formatada como 'HH:MM' indicando o horário atual em um relógio de 24 horas. (Então $0 \leq HH \leq 23$ e $0 \leq MM \leq 59$ e '00:00' segue "23:59").

Saída

Para cada tentativa, imprima uma string indicando o próximo horário palíndromo (excluindo o horário atual) em uma única linha formatada como 'HH:MM'. Ao determinar se HH:MM é palíndromo, ignore todos os zeros à esquerda em HH. Se HH for zero, então ignore todos os zeros à esquerda em MM.

Exemplo de entrada	**Saída para o exemplo de entrada**
3 00:00 23:30 14:59	00:01 23:32 15:51

Telefone Celular (azul)

Problema
B

Arquivo fonte: *B.c*, *B.cpp*, *B.java* ou *B.py*

Uma equipe de pesquisadores está desenvolvendo uma nova tecnologia para economizar tempo ao digitar mensagens de texto em dispositivos móveis. Eles estão trabalhando em um novo modelo que tem um teclado completo, assim os usuários podem digitar qualquer letra pressionando a tecla correspondente. Desta forma, um usuário precisa pressionar P teclas para digitar uma palavra de comprimento P.

No entanto, isto não é suficientemente rápido. A equipe vai montar um dicionário de palavras comuns que um usuário pode digitar. O objetivo é reduzir o número médio de teclas pressionadas necessárias para digitar palavras que constam no dicionário. Durante a digitação de uma palavra, sempre que houver apenas uma possibilidade para a seguinte letra, o sistema de telefone celular irá introduzi-la automaticamente, sem a necessidade de digitação. Para ser mais preciso, o comportamento do sistema de telefone celular irá ser determinado pelas seguintes regras:

1. O sistema nunca irá adivinhar a primeira letra de uma palavra, ou seja, para a primeira letra sempre terá que ser pressionada a tecla correspondente.

2. Se uma sucessão não-vazia de letras $c_1c_2...c_n$ for introduzida, e houver uma letra c tal que cada palavra no dicionário que começa com $c_1c_2…c_n$ também começa com $c_1c_2...c_nc$, em seguida, o sistema coloca a entrada c automaticamente, sem a necessidade de uma combinação de teclas. Caso contrário, o sistema aguarda o usuário.

Por exemplo, se o dicionário é composto das palavras “hello”, “hell”, “heaven” e “goodbye”, e o usuário pressiona “h”, o sistema colocará a letra “e” automaticamente, porque cada palavra que começa com “h” também começa com “he”. No entanto, uma vez que existem palavras que começam com “hel” e com “hea”, o sistema precisa agora esperar a próxima digitação do usuário. Se o usuário pressionar então o “l”, obtendo-se a palavra parcial “hel”, o sistema de entrada incluirá um segundo “l” automaticamente. Quando se tem o “hell” como entrada, o sistema não pode supor nada, porque é possível que a palavra terminou, ou também é possível que o usuário pode querer pressionar “o” para obter “hello”. Desta forma, para digitar a palavra “hello” o usuário precisa de três teclas, “hell” exige duas e “heaven” também requer duas, porque quando a entrada é “hea” o sistema pode colocar automaticamente o restante da palavra aplicando repetidamente a segunda regra. Da mesma forma, a palavra “goodbye” precisa de apenas uma tecla, porque depois de pressionar a inicial “g”, o sistema irá preencher automaticamente a palavra inteira. Neste exemplo, o número médio de teclas necessário digitar uma palavra no dicionário é, então, (3 + 2 + 2 + 1) / 4 = 2.00.

Dado um determinado dicionário, sua tarefa então é calcular o número médio de teclas necessárias para escrever uma palavra no dicionário com o novo sistema para celular desenvolvido pelos pesquisadores.

Entrada

Cada caso de teste é descrito por várias linhas. A primeira linha contém um número inteiro N representando o número de palavras no dicionário ($1 \leq N \leq 10^5$). Cada uma das próximas N linhas contém uma string não-vazia de no máximo 80 letras minúsculas do alfabeto inglês, representando uma palavra no dicionário. Dentro de cada caso de

teste todas as palavras são diferentes, e a soma dos comprimentos de todas as palavras é, no máximo, 10^6.

Saída

Para cada caso de teste de entrada imprima um número com duas casas decimais (arredondado caso necessário) que representa o número médio de pressionamentos de tecla necessários para digitar uma palavra no dicionário.

Exemplo de entrada	**Saída para o exemplo de entrada**
4 hello hell heaven goodbye 3 hi he h 7 structure structures ride riders stress solstice ridiculous	2.00 1.67 2.71

What is the class? (verde)

Problema C

Arquivo fonte: *C.c*, *C.cpp*, *C.java* ou *C.py*

You must implement a program that takes as input a series of *M* points in a 2-dimensional space, each classified by a numerical category. In addition to the points, you will be given an extra point for which you must determine the closest class, using the specified distance metric.

If there is a tie in the distance between the given point and more than one class, the class with the lowest numerical value should be chosen.

Euclidean Distance Formula:

$$d(Euc) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Manhattan Distance Formula:

$$d(Man) = |x_1 - x_2| + |y_1 - y_2|$$

Input

The entry will consist of the following elements:

First line: an integer N, where N is the number of points, followed by a letter T, which can be:

M: Use Manhattan distance.

E: Using Euclidean distance.

Next *N* lines: Each line contains 2 integers representing the coordinates of a point, followed by an integer representing the class of that point.

Last line: An additional point with 2 integers representing the coordinates of the point to be classified.

Output

The program must display a single integer representing the class of the closest point to the query point, according to the distance metric specified. In the event of a tie, display the class with the lowest value.

Restrictions

- $1 \leq N \leq 10^3$
- The coordinates of the points are integers between -10^3 e 10^3.

Input example	Output for input example
3 M 1 2 1 3 4 2 5 6 3 2 3	1

Explanation

The query point is (2,3) and the distance is Manhattan. The Manhattan distances to the points are:

- Distance to (1,2): $|2-1| + |3-2| = 1+1 = 2$
- Distance to (3,4): $|2-3| + |3-4| = 1+1 = 2$
- Distance to (5,6): $|2-5| + |3-6| = 3+3 = 6$

Classes 1 and 2 have the smallest distance (2), but class 1 is chosen because it is numerically smaller.

Challenging Hike (vermelha)

Problema D

Arquivo fonte: *D.c, D.cpp, D.java* ou *D.py*

Finally, the exhausting exams are behind you, and the time has come to relish a well-deserved vacation away from the college hustle. With your bags neatly packed, you set out for an adventure to explore one of the most renowned mountains in your region – a destination you've been dreaming of visiting for a considerable period.

This majestic mountain boasts impressive dimensions and offers a plethora of hiking options. Comprising a total of N landmarks, each uniquely identified by a number ranging from 1 to N, the mountain presents a captivating network of $N-1$ interconnected paths. These pathways ensure that you can effortlessly traverse from any landmark to another, connecting the entire mountain in a seamless web.

Each landmark, denoted by the index i, possesses an associated score, aptly named v_i. This score reflects the expected number of likes your social media post of that landmark would attract. Brimming with enthusiasm, you've decided to infuse an extra layer of excitement into your hike by embracing the "photos hype challenge". In this challenge, your objective is nothing short of extraordinary: capture and post photos in a manner that guarantees each successive photo garners more likes than its predecessor.

The rules of this challenge dictate your journey to unfold as follows:

1. You commence your hike from landmark 1.
2. Progressing along the available paths, you navigate through the landmarks, always moving forward and never backtracking.
3. At each landmark you visit, you are presented with the choice of capturing a photo and posting it on your social media platform.

As your journey unfolds, you're curious about the potential outcomes. Specifically, for every landmark i, you seek to determine the maximum number of photos that can be posted if you initiate the hike from landmark 1 and conclude at landmark i (although you not necessarily took a photo of landmark i). The challenge is to strategically select the landmarks to capture photos, ensuring that each photo outshines its predecessor in terms of expected likes.

Your task is to calculate this maximum number of photos for each landmark. Can you rise to the challenge and uncover the most captivating and likable path through this mountain?

Input

The first line contains an integer N ($1 \leq N \leq 10^5$), representing the number of landmarks in the mountain. The second line contains $N-1$ integers $p_2,p_3,\ldots,p_N$ ($1 \leq p_i \leq N$), where p_i represents that there is a path between the landmarks i and p_i. The third line contains N integers $v_1,v_2,\ldots,v_N$ ($1 \leq v_i \leq 10^9$), where v_i represents the expected number of likes a photo of the i-th landmark would get.

Output

Output a single line with $N - 1$ integers, where the i-th integer represents the maximum number of photos you can post if you start the hike on landmark 1 and finish on landmark $(i + 1)$.

Input example 1	**Output for input example 1**
5 1 1 3 3 5 7 7 6 8	2 2 2 3
Input example 2	**Output for input example 2**
5 3 1 3 1 5 4 7 6 5	2 2 2 1

O Diferente **(branca)**

Problema
E

Arquivo fonte: *E.c*, *E.cpp*, *E.java* ou *E.py*

Você recebe três dígitos *a*, *b*, *c*. Dois deles são iguais, mas o terceiro é diferente dos outros dois. Encontre o valor que ocorre exatamente uma vez.

Entrada

A primeira linha contém um único inteiro t ($1 \le t \le 270$) — o número de casos de teste.

A única linha de cada caso de teste contém três dígitos a, b, c ($0 \le a, b, c \le 9$). Dois dos dígitos são iguais, mas o terceiro é diferente dos outros dois.

Saída

Para cada caso de teste, imprima o valor que ocorre exatamente uma vez.

Exemplo de entrada	**Saída para o exemplo de entrada**
10 1 2 2 4 3 4 5 5 6 7 8 8 9 0 9 3 6 3 2 8 2 5 7 7 7 7 5 5 7 5	1 3 6 7 0 6 8 5 5 7

Doces do Dia das Bruxas (rosa)

Problema F

Arquivo fonte: *F.c*, *F.cpp*, *F.java* ou *F.py*

Todos os anos há o mesmo problema no Halloween: Cada vizinho só está disposto a dar certo número total de doces neste dia, não importa quantas crianças peçam, por isso pode acontecer que uma criança fique sem nada, se for tarde demais. Para evitar conflitos, as crianças decidiram que vão colocar todos os doces juntos e depois dividi-los igualmente entre si. A partir da experiência do Halloween do ano passado, elas sabem quantos doces recebem de cada vizinho. Visto que elas se preocupam mais com a justiça do que o número de doces que recebem, elas querem selecionar um subconjunto de vizinhos para visitar, para que na partilha cada criança receba o mesmo número de doces. Elas não vão ficar satisfeitas se sobrar doces que não possam ser divididos.

Seu trabalho é ajudar as crianças e apresentar uma solução.

Entrada

A entrada contém vários casos de teste.

A primeira linha de cada caso de teste contém dois inteiros c e n ($1 \leq c \leq n \leq 100000$), sendo o número de crianças e o número de vizinhos, respectivamente. A próxima linha contém n inteiros separados por espaço $a_1,..., a_n$ ($1 \leq a_i \leq 100000$), onde a_i representa o número de doces que as crianças recebem se visitarem vizinho i.

O último caso de teste é seguido por dois zeros.

Saída

Para cada caso de teste, imprima uma linha com os índices dos vizinhos que as crianças devem selecionar (aqui, o índice i corresponde ao vizinho i que dá um total de doces a_i). Se não houver solução, onde cada criança recebe pelo menos um doce, imprima "no sweets". Observe que, se existir várias soluções onde cada criança recebe pelo menos um doce, imprima o primeiro subconjunto encontrado.

Exemplo de entrada	**Saída para o exemplo de entrada**

`4 5`
`1 2 3 7 5`
`3 6`
`7 11 2 5 13 17`
`0 0`
	`2 3 4`
	`1 2`